

52 – Manipulação de Arquivos

Arquivos

DBA, Analista, Cientista ou Engenheiro de Dados:
DATALAKES (estruturados (**mysql**) ,semi-estruturados (**json**), não estruturados (**imagem**))

- .docx (Microsoft Word)
- .xlsx (Microsoft Excel)
- .html (Criar um site)
- .css (Estilizar um site)
- .js (Interatividade do site)
- .xml (Dados e Tags)
- .json (Dados (o.o))
- .parquet (dados em big data)
- .csv (dados separados por vírgulas)
- .txt (dados em formato de texto)
- .java (um arquivo java)
- .sql (scripts ddl , dml, dql, dcl ou dtl)

- **.xlsx (Microsoft Excel)**
- **.xml (Dados e Tags)**
- **.json (Dados (o.o))**
- **.parquet (dados em big data)**
- **.csv (dados separados por vírgulas)**
- **.txt (dados em formato de texto)**
- **.sql (scripts dml (data manipulation language))**

.txt

Arquivo de Texto

```
1  import java.io.FileWriter;
2  import java.io.IOException;
3  import java.time.LocalDateTime;           // data E hora juntos
4  import java.time.format.DateTimeFormatter; // moldura do formato
```

Run | Debug

```
void main(){  
    int r=0;  
    do{
```

```
20     try (FileWriter arquivo = new FileWriter(fileName: fileName: "diario.txt", append: append: true)) { // true = NÃO apagar, adicionar!
21         arquivo.write "[" + carimbo + "]" + mensagem + "\n"; // 3 carimbo + mensagem + pula linha
22         IO.println("✓ Registrado: [" + carimbo + "]" + mensagem);
23         IO.println(obj: obj: "Deseja registrar nova mensagem 1-sim 0-não: ");
24         r = Integer.parseInt(IO.readLine());
25     } catch (IOException e) {
26         IO.println("✗ Erro ao salvar no diário: " + e.getMessage());
27     }
28
29
30     }while(r==1);
31
32 }
33 }
```

CRUD

1 –Criar o
arquivo

2- Escreve no
Arquivo

3- Le o arquivo

4- Modifica

53- Dicionários

A Origem: O Java 1.0 e a classe Dictionary

Quando o Java foi lançado na versão **JDK 1.0 (1996)**, a estrutura oficial para armazenar dados no formato **chave-valor** era a classe abstrata `java.util.Dictionary`

Sua principal implementação padrão era a famosa `java.util.Hashtable`.

O `Dictionary` funcionava, mas carregava sérios problemas de arquitetura

Era uma classe, não uma interface: Isso quebrava o princípio de design de "programar voltado para interfaces".

Se você quisesse criar um dicionário customizado, era obrigado a estender essa classe abstrata, limitando a herança múltipla do Java

Sincronização forçada: A `Hashtable` era totalmente sincronizada (thread-safe) internamente.

Isso significava que, mesmo se você estivesse usando o código em uma aplicação simples e de thread única, seu programa sofria uma severa penalidade de performance devido aos bloqueios de concorrência (`locks`) desnecessários

Falta de visões colecionáveis: Era difícil iterar ou extrair facilmente apenas as chaves ou apenas os valores de forma padronizada

No ecossistema moderno do **Java 26**, a palavra "dicionário" no sentido conceitual de chave-v

alor continua totalmente sob o domínio da interface `Map`.

A classe original `java.util.Dictionary` segue mantida no JDK estritamente para manter retrocompatibilidade com códigos legados de 30 anos atrás, mas sem novas atualizações

Class HashMap<K,V>



```
graph BT; java.util.HashMap --> java.util.AbstractMap; java.util.AbstractMap --> java.lang.Object
```

java.lang.Object TODA CLASSE É **FILHA**

java.util.AbstractMap<K,V>

java.util.HashMap<K,V>

```
public class HashMap<K, V>
```

```
extends  AbstractMap<K, V>
```

```
implements Map<K, V>, Cloneable
```



```
3 public class Estudante {  
31     }  
32  
33     @Override  
34     public String toString() {  
35         return "Nome: " + nome + " | Curso: " + curso + " | Ano: " + ano;  
36     }  
37 }  
38
```

IMPRIMIR UM OBJETO (DADOS)

POLIMORFISMO – SOBRESCRITA

Sobrescreve o método do pai

```
@Override  
public String toString() {  
    return "Nome: " + nome + " | Curso: " + curso + " | Ano: " + ano;  
}
```

```
15 c class Algoritmo53A {
16     void main(){
51         estudantes.put(key: "MAT-1232", e10);
52         Estudante e11 = new Estudante(nome: "Gabriel I
53         estudantes.put(key: "MAT-1233", e11);
54         Estudante e12 = new Estudante(nome: "Thalita",
55         estudantes.put(key: "MAT-1234", e12);
56         estudantes.put(key: "MAT-1235", new Estudante(n
57         // Listar todos os estudantes cadastrados
58         for (Estudante e : estudantes.values()) {
59             IO.println(e);
60         }
61     }
```

// Toda a classe object
Map<^KString, Estudante> e



```
for (Estudante e : estudantes.values()) {  
    IO.println(e);  
}
```

- **HashMap (embaralhado)**
- **HashSet (inserção)**
- **TreeSet/TreeMap (ordenada)**